

Linking Software Process Modeling with Markov Decision Theory

Frank Padberg
Fakultät für Informatik
Universität Karlsruhe, Germany
padberg@ira.uka.de

Abstract. *Scheduling is a notoriously difficult task for managers of software projects. The time needed to complete some development activity in a software project is hard to estimate; feedback between activities causes rework and delays.*

In this paper, we present a dynamic software process model and its formalization as a discrete-time Markov decision process. The model focuses on capturing how the feedback between development activities affects the cost and schedule of a software project. The model explicitly represents scheduling actions, component coupling, and rework caused by design changes.

We use the MDP model to compute optimal strategies for several sample projects and analyze the resulting optimal strategies.

1. Introduction

This paper describes a stochastic scheduling model for software projects and its formalization. We have already presented this model in detail [2, 4, 5], but the implementation of the corresponding schedule optimization algorithms, as well as the sample computations shown in a later section, were completed just recently. The particular way in which our scheduling model describes feedback between concurrent activities seems to be novel in Operations Research [3, 7].

In our model, the software is developed by *teams* who work in parallel on the software's *components*. At any time, each team works on at most one component. The assignment between components and teams may change dynamically: *scheduling* takes place.

From time to time, a team will detect a defect, an inconsistency, or some other "problem" in the software's high-level design. Since the components are coupled, for example, through common interfaces, the necessary redesign will force a number of components to be *reworked*. Therefore, the teams do not work independently: the progress of one task not only de-

pends on the productivity of the corresponding team, but also on the progress of the other tasks. This kind of feedback is typical for software projects and is a key driver for the uncertainty about the future progress of a software project.

Each project advances through so-called *phases*. By definition, a phase ends when one or more teams finish their current task, or, when the software's high-level design gets revised because of a design problem. The phases in our model should not be confused with the development phases in other process models, such as the waterfall model.

Scheduling actions take place only at the end of a phase. The rationale behind this restriction is that it does not make sense to re-schedule a project as long as nothing unusual happens. At the end of a phase, staff is available again for allocation, or re-scheduling the project might be appropriate because of some design changes.

2 Markov Decision Model

In this section, we formalize the software process as a Markov decision process (MDP). The software process *stochastically* moves between states over time. In each state, a scheduling action is chosen. Once an action has been chosen, the transition to the next state is governed by transition probabilities. Each transition bears some cost. An optimal scheduling strategy minimizes the expected project cost.

2.1. States

The *state* of the project changes at the end of each phase. The state ζ by definition consists of four parts: a progress vector ζ^P ; a rework vector ζ^R ; an assignment vector ζ^A ; and a countdown ζ^C . Since time is discrete in the model, the entries in these vectors are integers.

The *progress* ζ_k^p of component k is defined as the *net* development time that has been spent working on the component. The net development time is obtained from the total development time by subtracting all rework times which were spent on adapting the component to high-level design changes.

The *rework time* ζ_k^r for component k is the time that yet must be spent with adapting the component to high-level design changes. As soon as a component's rework time has been counted down to zero, development of the component can resume as planned and net progress is achieved again.

Entry ζ_k^a in the *assignment vector* is the number of the team which has worked on component k most recently. The entry equals 0 if none of the teams has worked on the component yet.

The *countdown* ζ^c is the time left until the project's deadline will be reached. The project ends when all components are completed (including all rework) or the deadline is passed.

2.2. Actions and strategies

A scheduling action is modelled as an *action vector* which has one entry for each team. The action a_i for team i is the number of the component to which the team is allocated during the next phase. The entry is set to -1 if the team is stopped.

Actions may depend on the current state of the project. A *scheduling strategy* or *policy* is a function which (deterministically) specifies an action for each project state and phase. Simple scheduling strategies use a fixed priority list for the components. With list policies, the currently available teams are allocated to the next unprocessed components in the list.

Any team may be scheduled to work on any component, but the teams need not be equally qualified. Different skill levels of the teams are modelled using different probability distributions for their component completion times, see subsection 3.1.

2.3. Transition probabilities

Given a project state ζ and a scheduling action a , the next state of the project will be η only with a certain probability, the *transition probability* $P(\zeta, a; \eta)$. The transition probability does not depend on any information about the project's history except what is coded in the current state ζ . The resulting process $\zeta(0), a(0), \zeta(1), a(1), \dots$ is called a *Markov decision process* [1, 6].

To compute the transition probabilities, statistical data about past projects and high-level design data about the software product are required as input. The

statistical data are a measure of the pace at which the teams have made progress in past projects. For a given team and component, the so-called *base distributions* describe how likely it is that the team will complete the component or report a design problem after a certain (net) development time. The required design data are a measure for the strength of the coupling between the software's components. The stronger the coupling is the more likely it is that problems originating in one component will lead to rework in other components (ripple effects). The coupling strength must be computed from the high-level design of the software. For more details on the definitions see [2, 4, 5].

2.4. Cost functions

Associate with the transition from state ζ to some state η the *transition cost* $g(\zeta, a; \eta)$. The cost of a transition depends on the scheduling action a chosen.

In the software process, the transition cost may be the duration $d = \zeta^c - \eta^c$ of the phase which passes between the two states, or the staffing cost for the phase. Each state is also assigned a *terminal cost* $g(\zeta)$ which is incurred when ending the process in that state. In our case, the terminal cost of a state which corresponds to project failure (deadline exceeded) is some high penalty; terminal states which correspond to a successful project have zero terminal cost.

Using the transition costs, one can assign to any finite sequence

$$\begin{aligned} \omega &= \zeta(0), a(0) \dots \\ &\dots \zeta(m-1), a(m-1), \zeta(m) \end{aligned}$$

of state-action pairs its cost

$$g(\omega) = \sum_{i=0}^{m-1} g(\zeta(i), a(i); \zeta(i+1))$$

by summing up the costs of all the transitions in the sequence. The sequence ω can be viewed as the *path* or the *course* of the project when observed for a period of m phases from state $\zeta(0)$ on. The probability that the process will proceed according to the sequence ω is equal to the product

$$P(\omega) = \prod_{i=0}^{m-1} P(\zeta(i), a(i); \zeta(i+1))$$

of the corresponding transition probabilities.

A software project starts in the initial state σ (zero net progress, zero rework) Since a project must finish before the deadline is exceeded, a project will last for at most x_0 phases.

2.5. Optimal strategies

In a stochastic setting, the best one can expect to get is a *stochastically* optimal strategy, that is, a strategy which minimizes the *expected* project cost. Hence, schedule optimization for software projects amounts to computing a strategy which has minimal expected cost in our Markov decision model, where the cost function g is either the development time function or the staffing cost function.

The search space for this optimization problem is far too huge to perform a full search. More efficient algorithms exist, such as Backwards Dynamic Programming, Value Iteration, and Policy Iteration, see [1, 6].

3. Sample Computations

In this section, we use our Markov decision model to compute the optimal scheduling policy for a small software project under various settings. We analyze the scheduling decisions of the resulting optimal strategies.

In all examples, *cost means "project duration."*

3.1. Input data

The sample project has four components A, B, C, and D. There are two teams, One and Two, working simultaneously on the project. We use shifted binomial distributions for the base probabilities. Differences between the components with respect to their complexity and risk-level are modelled in the following way:

- The distributions for the net development times have different means and variances, see TABLE 1. The expected net effort for A and B is smaller than for C and D. The distributions for C and D have a higher variance than for A and B.

Table 1. Mean, variance, and risk level of base probabilities (no specialization).

component	teams	mean	variance	risk
A	One, Two	4.0	1.0	0.10
B	One, Two	4.0	1.0	0.20
C	One, Two	6.5	1.5	0.20
D	One, Two	6.5	1.5	0.30

- The probability that design problems will originate from a component varies from component to component. The risk is lowest for component A and highest for component D, see TABLE 1.
- Specialization of a team on a certain component is expressed by using base distributions which have a smaller mean as compared to a non-specialized team, see TABLE 2. In case of specialization, team

One is faster on components A and C, and team Two is faster on components B and D.

Table 2. Mean, variance, and risk level of base probabilities for specialized teams.

component	team	mean	variance	risk
A	One	2.0	1.0	0.10
A	Two	4.0	1.0	0.10
B	One	4.0	1.0	0.20
B	Two	2.0	1.0	0.20
C	One	3.5	1.5	0.20
C	Two	6.5	1.5	0.20
D	One	6.5	1.5	0.30
D	Two	3.5	1.5	0.30

We also vary the strength of the coupling between the components: minimal, uniform, and maximal.

- Minimal coupling means that only those components in which a design problem occurs will have to be reworked, but no other components. With minimal coupling, tasks are being worked on independently of each other. This is the best-case scenario with no feedback between different tasks.
- Maximal coupling means that always all components will have to be reworked in case of a redesign, no matter in which component a design problem occurs. This is the worst-case scenario with maximal feedback between different tasks.
- Uniform coupling assumes no prior knowledge about the coupling between the components. Each set of components bears the same risk of being affected by rework in case of a redesign, no matter in which component a design problem arises.

3.2. Implementation

Since our software process MDP has no cycles, it is possible to combine Value Iteration with a depth-first search for accessible states; this yields a much faster algorithm. We have implemented this algorithm in C#. The results are exact up to the precision of the floating point arithmetic. TABLE 3 shows the CPU time required for computing the optimal policy (min:sec) and the number of different states of the MDP for our sample project with no specialization:

Table 3. Number of MDP states and CPU time for schedule optimization (no specialization).

coupling	minimal	uniform	maximal
MDP states	2,709	301,355	90,335
CPU time	0:01	6:25	0:47

The computations were carried out on a Pentium III processor with 700 MHz clock rate and 256 MB of

main memory. The numbers are similar if the project teams are specialized.

3.3. Optimal cost

FIGURE 1 shows how the optimal cost (= duration) of the sample project varies with the strength of the coupling between the components and with the degree of specialization of the teams on components.

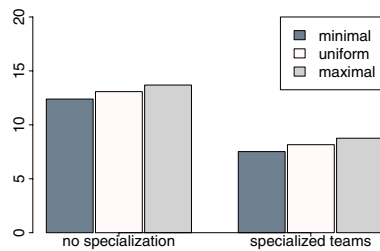


Figure 1. Optimal duration for sample project.

The optimal project cost grows with the strength of the coupling between the components. The stronger the coupling, the more often and widely will rework spread out during the project.

The optimal cost is lower in case of specialized teams than without specialization: an optimal policy takes advantage of the fact that specialized teams have a shorter expected net development time for the components than non-specialized teams, see the next subsection.

3.4. Typical schedules

To visualize the schedules which actually occur for an optimal policy we use special Gantt charts which are computed from the *mean* net development times and the *mean* rework times for each component. These numbers are collected from *simulation traces* for the policy. For each project setting, we have simulated 10,000 full project trajectories for the optimal policy. FIGURE 2 shows a typical "average schedule" for the optimal policy in the sample project with no specialization and uniform coupling. The numbers below the bars are the mean development times for the components (including rework).

The optimal policy first assigns component C to team One and component D to team Two. Team One finishes earlier than team Two; thus, the optimal policy assigns component B to team One. The next team to finish is team Two, which finally is assigned to the last unfinished component, A. The simulations show that the project will take this path with about 61 percent probability.

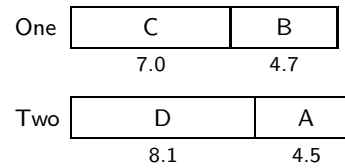


Figure 2. First typical schedule for the optimal policy (no specialization, uniform coupling).

There is another typical path which the project can take, see FIGURE 3. The optimal policy assigns the components C and D as described before; but now, the first team to finish its current task is team Two. Therefore, the optimal policy assigns component B to team Two instead of team One. Finally, component A is assigned to team One. The project will take this alternative path with about 38 percent probability.

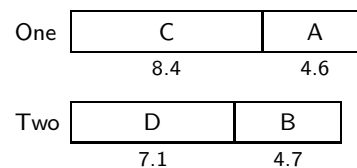


Figure 3. Second typical schedule for optimal policy (no specialization, uniform coupling).

In case of specialized teams, the Gantt charts (not shown here) yield that the optimal policy manages to *always* assign the teams to their favorite components: team One works on components A and C, team Two on components B and D.

References

- [1] Bertsekas: *Dynamic Programming and Optimal Control*. Athena Scientific, 1995
- [2] Padberg: "Scheduling Software Projects to Minimize the Development Time and Cost with a Given Staff", Proceedings APSEC 8 (2001) 187-194
- [3] Padberg: "A Stochastic Scheduling Model for Software Projects", Dagstuhl Seminar on Scheduling in Computer and Manufacturing Systems, June 2002
- [4] Padberg: "Using Process Simulation to Compare Scheduling Strategies for Software Projects", Proceedings APSEC 9 (2002) 581-590
- [5] Padberg: "A Discrete Simulation Model for Assessing Software Project Scheduling Strategies", Int. Journal on Software Process Improvement and Practice SPIP 10:3-4 (2002) 127-139
- [6] Ross: *Introduction to Stochastic Dynamic Programming*. Academic Press 1983
- [7] Weglarz: *Project Scheduling. Recent Models, Algorithms, and Applications*. Kluwer, 1999